

Document de Production - Voitures déformables

Adam HAMOUCH

February 2026

1 Introduction

Ce rapport documentera les recherches qui entoureront mon projet de création de voitures déformables, il aura pour vocation moins de décrire l'implémentation du projet tech art que de détailler les différentes recherches qui y mènent, ainsi que des informations qui ne sont pas forcément en lien avec le projet en question.

2 Skeletal vs Static vs Dynamic vs Procedural mesh

Afin de représenter notre déformation de voiture le premier choix qui s'offrait à moi était celui du type de mesh à choisir pour les véhicules, parmi ceux-ci il y a le skeletal mesh, basé sur un squelette animé il est très délicat de le modifier/déformer en runtime, cela impliquerait une régénération profonde du mesh et de son squelette.

Il y a également l'option du static mesh, bien que par principe on s'en sert principalement pour représenter des objets inanimés son usage est pertinent dans notre cas, en passant par du shader programming on peut donner cet effet de déformation sans modifier les valeurs du mesh en mémoire préservant ainsi les performances et la sécurité du programme.

Quant au dynamic mesh il apparaît d'abord comme une solution intéressante en raison du fait qu'il soit pensé pour être modifiable, cependant il est davantage approprié aux modifications éditeur que runtime, du fait d'un rebuild cpu assez long.

Pour finir on a également le Procedural mesh, dans un cas où nous voulions appliquer des modifications profondes au mesh il aurait été le choix idéal, similaire au static mesh il permet des modifications en runtime et rebuild des meshes, la différence fondamentale entre les deux réside donc dans la méthode utilisée (dynamic/static draw) à l'avantage du static dans la mesure où les infos ne sont pas update à chaque frame.

Au final en raison des besoin du projet et de l'importance donnée aux performances mon choix s'est porté sur le static mesh.

3 CPU deformation vs Vertex shader

Comme évoqué précédemment, la déformation d'un mesh côté CPU pose des problèmes de performances importants pour une plus-value limitée dans le contexte de ce projet. En effet, ce type de déformation implique la modification directe des données de géométrie du mesh, notamment les buffers de vertices, ce qui nécessite un ré-envoi de ces données vers le GPU. Cette opération engendre des coûts significatifs en termes de calcul et de synchronisation entre le CPU et le GPU, particulièrement dans un contexte où les impacts sont fréquents.

De plus, la reconstruction partielle ou totale du mesh peut introduire des risques de désynchronisation dans le pipeline de rendu, tout en compliquant la gestion de la mémoire et du multithreading. Ainsi, au regard des contraintes de performance et de stabilité du moteur, la déformation CPU n'apparaît pas comme une solution pertinente pour ce projet, dont l'objectif est avant tout de produire une déformation visuelle crédible sans altérer les données géométriques sous-jacentes.

Dans cette optique, le choix s'oriente vers une approche basée sur la modification du vertex shader. Le vertex shader est responsable du traitement des données de chaque vertex, notamment leur position, leurs normales et leurs attributs associés, avant leur projection à l'écran. En intervenant à ce niveau du pipeline graphique, il est possible de modifier la position finale des vertices sans altérer les données du mesh stockées en mémoire.

Dans Unreal Engine, le vertex shader est accessible et configurable via le Material Graph, qui permet de définir des comportements personnalisés à l'aide de nœuds ou de code HLSL. Cette approche offre une grande flexibilité tout en conservant des performances élevées, puisque les calculs sont effectués entièrement sur le GPU.

Ainsi, la déformation est uniquement visuelle et repose sur des données auxiliaires transmises au shader, telles que des paramètres de matériau, des vertex colors ou des textures, permettant de simuler des impacts localisés tout en préservant l'intégrité du mesh original et la cohérence du pipeline de rendu.

4 World Position offset

Le world position offset est donc une des variables contenues dans le vertex shader, modifiable dans le material graph par node/hlsl elle permet de manipuler la position rendue des vertices de l'objet auquel le material est attaché.

Dans notre cas on doit porter une attention particulière à la valeur qu'on attribue à WPO, en effet donner un vecteur arbitraire ne déformera pas l'objet de manière réaliste, dans ce sens des outils comme le `vertexnormalws` permettent de déterminer l'orientation des sommets de l'objets pour les déformer en conséquence, on se basera aussi sur la vitesse du véhicule mais nous y reviendrons plus tard

Un autre outil très intéressant dans le material graph est le `custom`, il consiste en un node lié à un fichier dans lequel on peut coder une fonction, souvent une formule mathématique, qui sera compilée et dont la valeur de sortie peut être utilisée. On peut y éditer les valeurs d'entrée et de sortie depuis l'onglet `details`.

Le processus s'effectuant avant le lighting d'unreal on peut constater des ombres ou rendu lumineux incohérent sur les objets soumis à cette modification, de ce fait il peut être important de recalculer les normals en même temps.

5 Vertex color et UV storage

Le vertex color est un autre outil d'autant plus intéressant dans notre cas, il permet d'appliquer plusieurs materials au sein d'un seul, afin de par exemple rendre une montagne avec une texture de roche sur laquelle on rends une texture de neige aux sommets. Pour se faire on peut autant utiliser des textures que, comme son nom l'indique, des couleurs. On utilise communément un lerp pour jouer entre les différents "états".

Dans le cas de ce projet en particulier le Vertex color peut avoir un usage très important, il consiste en fait en une valeur de stockage de 4 float modifiables en runtime et qui communique directement avec le gpu.

6 Runtime virtual texture et Render target

Nous avons vu la fonction des vertex color dans Unreal engine, cependant dans notre cas ils comportent une limite : ils sont difficilement éditables en runtime, c'est pourquoi nous explorerons deux solutions plus adaptés à ce projet : Runtime virtual texture et Render target.

Nous avons donc d'abord le `rvt`, comme son nom l'indique il s'agit d'une texture modifiable en runtime, elle s'applique aux objets sur lesquels elle a été assignée et qui sont contenu dans un volume. Communément elle peut servir à dessiner par dessus une texture en runtime. Son intérêt spécifique réside en sa précision spatiale, en effet les modifications appliquées à la texture se répliquent à un endroit précis sur l'objet. Dans notre cas la texture pourrait servir à stocker les données de dégâts pour les appliquer dans le shader aux endroits précis où la voiture a subi des chocs.

Les données RVT peuvent également être utilisés in editor, ses données per-

mettent de jouer sur un tas de situations, que ce soit pour faire apparaître, disparaître, jouer sur l'opacité, déformer ou dessiner des objets/sur des objets.

Cependant ce système a évidemment des limitations, il n'est pas conçu pour être constamment appelés au dépends des performances (la moindre modification implique de redraw toute la zone sous le volume) et ne permet pas d'appliquer des modifications additives, ce qui est problématique dans notre cas. En effet les changements appliqués à une rvt sont gravés dans le marbre, au contraire d'une render Target qui consiste en une addition appliquée à une forme.

C'est pour cela qu'on privilégiera ici l'usage des render target. En effet il s'agit d'une texture dynamique pouvant être appliquée à un objet précis et modifiée en runtime, dans notre cas on peut s'en servir comme mémoire tampon qui va enregistrer les impacts pour les concrétiser ensuite dans le shader. Cette solution est de ce fait plus stable dans notre cas bien qu'elle puisse être un peu plus complexe à mettre en place.

7 Morph Target

Au final la complexité d'intégration des méthodes précédentes m'ont poussée à opter pour les morphs target. Leur fonctionnement est très simple, sur blender on peut créer des états "clés" de déformation d'un objet, sur unreal dans le skeletal mesh on peut récupérer ces états dictés par un int de 0 à 1, dans notre code on peut trigger ces valeurs pour définir un état de déformation en fonction de l'intensité de l'impact (hit impulse normal) et de l'orientation du choc (inverse transform du point d'impact).